Bilgisayar Programlama

Fonksiyonlar — Temel Yapılar ve Modüler Programlamaya Giriş

İçindekiler

1. Fonksiyon Kavramına Giriş
   - Tanım, Gerekçe ve Modüler Tasarım Felsefesi
2. Standart Kütüphane Fonksiyonları
   - `stdio.h`, `math.h`, `stdlib.h` ve Hazır Araç Kullanımı
3. Fonksiyonun Anatomisi
   - Geri Dönüş Tipi, İsimlendirme Standartları, Parametreler, Gövde
   - `void` Fonksiyonlar ve Fonksiyon Çeşitleri

İçindekiler (Devam)

4. Parametre Aktarımı ve Geri Dönüş Mekanizmaları
   - Parametre — Argüman Ayrımı, Çoklu Parametre
   - Değer ile Çağırma (Call by Value) ve Bellek Modeli
   - `return` Semantiği, İzleme Tabloları, Erken Çıkış
   - İç İçe Fonksiyon Çağrıları ve Çağrı Yığını (Call Stack)
5. Fonksiyon Bildirimi (Prototip) ve Kapsam Kuralları
   - Prototip Zorunluluğu, Yerel ve Global Değişkenler
   - Blok İçi Kapsam, İsim Gölgeleme (Shadowing)
   - Fonksiyon Çağrısının Maliyeti (Overhead)
6. Sık Karşılaşılan Hatalar ve Çözümleri
7. Uygulamalı Laboratuvar Çalışmaları

BÖLÜM 1

Fonksiyon Kavramına Giriş

1.1 Fonksiyon Tanımı

Fonksiyon (Function): Belirli bir görevi yerine getirmek üzere tasarlanmış, isimlendirilmiş ve tekrar kullanılabilir (reusable) kod bloklarıdır.

Yazılım mühendisliğinde fonksiyonlar, **DRY (Don't Repeat Yourself)** ilkesinin temel uygulama aracıdır. Binlerce satırlık monolitik bir `main()` fonksiyonu yerine, her bir alt görevi kendi fonksiyonuna soyutlamak; kodun bakımını, test edilebilirliğini ve okunabilirliğini doğrudan iyileştirir.

Kazanım	Açıklama
Tekrar Kullanılabilirlik	Bir fonksiyon, programın farklı noktalarından istenildiği kadar çağrılabilir.
Soyutlama (Abstraction)	İç detaylar gizlenir; çağıran taraf yalnızca arayüzü (imzayı) bilir.
Hata Yalıtımı	Hata çıktığında arama alanı, ilgili fonksiyonun gövdesiyle sınırlanır.
İş Bölümü	Büyük projelerde farklı mühendisler farklı fonksiyonları bağımsız olarak geliştirebilir.

1.2 Monolitik (Tek Parça) Yaklaşımın Sorunları

Tüm mantığı tek bir `main()` bloğunda kodlamak (Monolitik yaklaşım), yazılımı sürdürülemez hale getirir.

× Kod Karmaşası:

```
main() {
  1. Sensörleri başlat
  2. Kameradan veri oku
  3. Engel algıla
  4. Açı hesapla
  5. Motorlara PWM gönder
  ...
  500+ satır daha
}
```

Sorun	Sonucu
Uzun Kod	Okumak ve anlamak zorlaşır.
Kopya Kod	Bir düzeltme her yerde yapılmalıdır.
Hata Arama	500 satırda hata bulmak zordur.
Takım İşi	Dosya üzerinde çakışmalar olur.

Özet: Karmaşıklığı yönetmek için modüler yaklaşım (fonksiyonlar) bir zorunluluktur.

1.3 Modüler Yaklaşım

Modüler yaklaşımda her alt görev, kendi fonksiyonuna soyutlanır. `main()` fonksiyonu yalnızca **üst düzey akış kontrolünü** yönetir.

✅ Modüler Yaklaşım — Görevler fonksiyonlara dağıtılır:

```
int main() {
    sistemleri_baslat();

    while (1) {
        float mesafe = ultrasonik_oku();
        float aci    = gyro_oku();
        motorlari_sur(mesafe, aci);
    }
    return 0;
}
```

Her bir satır karmaşık bir alt sistemi temsil eder; ancak `main()` içindeki kod, bir iş akış şeması kadar okunaklıdır.

1.4 Monolitik ve Modüler Karşılaştırma Tablosu

Her iki yaklaşımı karşılaştıran tablo aşağıda verilmiştir:

Kriter	× Monolitik	✅ Modüler
Okunabilirlik	Düşük — yüzlerce satır iç içe	Yüksek — her fonksiyon tek görev
Tekrar Kullanım	Mümkün değil — kod kopyala/yapıştır	Fonksiyon istenen yerde çağrılır
Hata Ayıklama	Zor — tüm dosyada arama	Kolay — yalnızca ilgili fonksiyon
Takım Çalışması	Çakışmalar kaçınılmaz	Her mühendis kendi fonksiyonunu yazar
Test Edilebilirlik	Bütünleşik test zorunlu	Fonksiyon bazında birim testi
Bakım Maliyeti	Yüksek	Düşük

Sonuç: Mekatronik yazılım projelerinde modüler yaklaşım bir tercih değil, **zorunluluktur.**

BÖLÜM 2

Standart Kütüphane Fonksiyonları

2.1 Hazır Fonksiyonların Yeniden Değerlendirilmesi

C dilini öğrenmeye başladığımızdan itibaren fonksiyonları farkında olmadan kullanmaktayız. `printf()`, `scanf()`, `sqrt()` gibi çağrılar, C standart kütüphanesi tarafından sağlanan hazır fonksiyonlardır.

```
// stdio.h kütüphanesinden — Çıkış donanımının detaylarını bilmemize gerek yoktur.
printf("Sistem hazir...\n");

// math.h kütüphanesinden — Karekök algoritmasını kendimiz uygulamayız.
float sonuc = sqrt(16.0);   // 4.0 döndürür
```

Hedef: Bu derste, standart kütüphane fonksiyonlarını kullanmanın ötesine geçerek kendi fonksiyonlarımızı tasarlama ve uygulama yetkinliğini kazanmak.

2.2 `math.h` Kütüphanesindeki Temel Fonksiyonlar

Mekatronik hesaplamalarında sıklıkla başvurulan hazır matematik fonksiyonları:

Fonksiyon	Açıklama	Örnek Çağrı	Sonuç
`sqrt(x)`	Karekök	`sqrt(144.0)`	12.0
`pow(x, y)`	Üs alma (x^y)	`pow(2.0, 10.0)`	1024.0
`fabs(x)`	Mutlak değer (double)	`fabs(-3.7)`	3.7
`ceil(x)`	Yukarı yuvarlama	`ceil(4.2)`	5.0
`floor(x)`	Aşağı yuvarlama	`floor(4.9)`	4.0
`sin(x)` / `cos(x)`	Trigonometri (radyan)	`sin(3.14/2)`	≈1.0

2.3 `math.h` Uygulama Örneği — Kuvvet Momenti Hesabı

Kuvvet momenti, bir kuvvetin bir eksen etrafındaki döndürme etkisini ifade eder.

- Formülü $M = F \cdot r \cdot \sin(\theta)$ şeklindedir.
- Burada F kuvvet, r kuvvet kolu, θ ise kuvvetin kuvvet kolu ile yaptığı açıdır.

```c
#include <stdio.h>
#include <math.h>  // Derleme: gcc dosya.c -lm

#define PI 3.14159265

int main() {
    double aciDerece = 45.0;
    double aciRadyan = aciDerece * PI / 180.0;  // Derece → Radyan dönüşümü

    printf("sin(45) = %.4f\n", sin(aciRadyan));              // 0.7071
    printf("Moment  = %.2f Nm\n", 15.0 * sin(aciRadyan));   // Kuvvet momenti hesabı
    return 0;
}
```

⚠️ **Derleme Notu:** `math.h` kullanan programlar `gcc dosya.c -lm` bayrağıyla derlenmelidir. Ayrıca `M_PI` sabiti bazı derleyicilerde varsayılan tanımlı gelmeyebilir; bu durumda `#define PI 3.14159` ...

2.4 Sık Kullanılan Diğer Kütüphaneler

Kütüphane	İçerdiği İşlevler	Örnek Fonksiyonlar
`stdio.h`	Standart Girdi/Çıktı	`printf()` , `scanf()`
`math.h`	Matematiksel İşlemler	`sqrt()` , `pow()` , `sin()`
`stdlib.h`	Araç Fonksiyonları	`abs()` , `rand()` , `srand()`
`time.h`	Zaman İşlemleri	`time()` , `clock()`

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
int main() {
    srand(time(NULL));                      // Rastgele sayı üretecini başlat
    int sensor_sim = rand() % 1024;         // 0-1023 arası rastgele ADC değeri
    printf("Simulasyon ADC: %d\n", sensor_sim);
    printf("Mutlak fark: %d\n", abs(sensor_sim - 512));
    return 0;
}
```

BÖLÜM 3

Fonksiyonun Anatomisi

3.1 Bir Fonksiyonun Yapı Taşları

Bir C fonksiyonu dört temel bileşenden oluşur:

1. **Geri Dönüş Tipi (Return Type):** Fonksiyonun çağırana ileteceği verinin türü (`int` , `float` , `double`). Değer döndürmeyen fonksiyonlar `void` olarak bildirilir.
2. **Fonksiyon Adı (Identifier):** İşlevini yansıtan, C isimlendirme kurallarına uygun tanımlayıcı.
3. **Parametre Listesi (Parameters):** Fonksiyonun dışarıdan alacağı girdi değişkenlerinin tür ve isim tanımları.
4. **Gövde (Body):** `{ }` süslü parantezler arasında yer alan, fonksiyonun iş mantığını barındıran blok.

```
// [1] Geri Dönüş Tipi  [2] Fonksiyon Adı  [3] Parametre Listesi
    float              alanHesapla       (float yaricap)
    {
        // [4] Gövde — İş mantığı
        float alan = 3.14159 * yaricap * yaricap;
        return alan;   // Hesaplanan değeri çağırana ilet
    }
```

3.2 Fonksiyon Yapısının Görsel Ayrıştırması

```
            ┌── Geri Dönüş Tipi
            │
            │      ┌── Fonksiyon Adı
            │      │
            ▼      ▼          ┌─ Parametre Listesi─┐
        ┌─────┐ ┌──────────┐ ┌──────────────────┐
        │float│ │alanHesapla│ │(float yaricap)   │
        └─────┘ └──────────┘ └──────────────────┘
        {                                        ← Gövde Başlangıcı
            float alan = 3.14 * yaricap * yaricap;
            return alan;     ← Geri dönüş değeri
        }                                        ← Gövde Sonu
```

Önemli Kurallar:

- Geri dönüş tipi ile gövdedeki `return` değerinin tipi uyumlu olmalıdır.
- Parametre listesi boşsa `void` yazılabilir: `void fonk(void)`
- Gövde en az bir satır içermelidir (boş gövde `{ }` geçerli ancak anlamsızdır).

3.3 İsimlendirme Standartları

Fonksiyon isimleri, gerçekleştirdikleri eylemi açık biçimde yansıtmalıdır:

İsimlendirme	Değerlendirme	Açıklama
`hesapla()`	× Belirsiz	Neyi hesapladığı belli değil.
`f1()`	× Anlamsız	Hiçbir bağlam sağlamaz.
`voltajHesapla()`	✅ CamelCase	Açık ve tanımlayıcı.
`motor_baslat()`	✅ snake_case	Gömülü sistemlerde yaygın tercih.

⚠️ C Dili Kısıtlamaları:

- Fonksiyon adlarında **boşluk** ve **Türkçe karakter** (ş, ğ, ı, ö, ü, ç) kullanılamaz.
- İsimler bir rakamla başlayamaz.
- C dilinin ayrılmış anahtar kelimeleri (reserved keywords) isim olarak kullanılamaz.

3.4 Değer Döndürmeyen (`void`) Fonksiyonlar

Bazı fonksiyonlar hesaplama sonucu üretmez; yalnızca bir eylem gerçekleştirir (ekrana yazma, donanıma sinyal gönderme vb.). Bu tür fonksiyonların geri dönüş tipi `void` olarak bildirilir.

```
// Parametresiz ve değer döndürmeyen bir fonksiyon
void acilDurumDurdur(void) {
    printf("!!! TEHLIKE !!!\n");
    printf("Tum motor gucleri kesildi!\n");
    // Motor_Role_Kapat();  // Donanım komutu
}

int main() {
    float mesafe = 5.0;  // cm

    if (mesafe < 10.0) {
        acilDurumDurdur();  // Fonksiyon çağrısı
    }

    return 0;
}
```

3.5 Fonksiyon Çeşitleri — Özet Tablosu

C dilinde fonksiyonlar **parametre alma** ve **değer döndürme** durumlarına göre dört temel kategoriye ayrılır:

Tür	Parametre	Return	Örnek İmza	Kullanım Alanı
Tür 1	×	×	`void bilgiYaz(void)`	Sabit mesaj yazdırma, LED yakma
Tür 2	✅	×	`void motorSur(int hiz)`	Donanım komutu gönderme
Tür 3	×	✅	`float sicaklikOku(void)`	Sensör okuma
Tür 4	✅	✅	`float voltajCevir(int adc)`	Hesaplama ve dönüştürme

Her bir türe ait örnekler ilerleyen slaytlarda ayrıntılandırılacaktır.

3.6 Tam Program Örneği — Dört Fonksiyon Türü Bir Arada

```c
#include <stdio.h>

// Tür 1: Parametresiz, değer döndürmeyen
void baslikYaz(void) {
    printf("=== Mekatronik Kontrol Sistemi ===\n");
}

// Tür 2: Parametreli, değer döndürmeyen
void ledYak(int pin) {
    printf("LED pin %d: ACIK\n", pin);
}

// Tür 3: Parametresiz, değer döndüren
int sicaklikOku(void) {
    return 72;   // Simülasyon: 72 derece
}

// Tür 4: Parametreli, değer döndüren
float voltajCevir(int adc_deger) {
    return (adc_deger * 5.0) / 1023.0;
```

BÖLÜM 4

Parametre Aktarımı ve Geri Dönüş Mekanizmaları

4.1 Parametre ve Argüman Kavramları

Bu iki terim sıklıkla birbirine karıştırılır; ancak teknik olarak farklı anlam taşırlar:

- **Parametre (Parameter):** Fonksiyonun **tanım** (definition) aşamasında bildirilen değişken isimleridir.
- **Argüman (Argument):** Fonksiyonun **çağrı** (call) aşamasında gönderilen **somut değerlerdir.**

```c
// x ve y → PARAMETRE (fonksiyon tanımında)
int topla(int x, int y) {
    return x + y;
}

int main() {
    int sayi1 = 5, sayi2 = 10;

    // sayi1 ve sayi2 → ARGÜMAN (çağrı anında gönderilen değerler)
    int sonuc = topla(sayi1, sayi2);
    printf("Sonuc: %d\n", sonuc);   // 15
    return 0;
}
```

4.2 Çoklu Parametre ve Farklı Veri Tipleri

Bir fonksiyon, birbirinden farklı veri tiplerinde birden fazla parametre alabilir. Gömülü donanım kontrol fonksiyonları genellikle bu yapıdadır.

```c
// Farklı veri tiplerinde üç parametre alan bir motor kontrol fonksiyonu
void dcMotorSur(int port_no, float hiz_pwm, char yon) {
    printf("Motor Port: %d aktif.\n", port_no);

    if (yon == 'I') {
        printf("Yon: Ileri, Guc: %.1f%%\n", hiz_pwm);
    } else {
        printf("Yon: Geri, Guc: %.1f%%\n", hiz_pwm);
    }
}

int main() {
    dcMotorSur(1, 80.5, 'I');   // Port 1, %80.5 güç, İleri yön
    dcMotorSur(2, 60.0, 'G');   // Port 2, %60.0 güç, Geri yön
    return 0;
}
```

4.3 Değer ile Çağırma (Call by Value) — Kavram

C dilinde fonksiyonlara argüman gönderildiğinde, orijinal değişkenin değil yalnızca **kopyasının** aktarıldığı unutulmamalıdır. Fonksiyon içindeki değişiklik **orijinal değişkeni etkilemez.**

```c
void ikiyeKatla(int sayi) {
    sayi = sayi * 2;  // Yalnızca kopya üzerinde işlem yapılır
    printf("Fonksiyon icinde: %d\n", sayi);  // 20
}

int main() {
    int x = 10;
    ikiyeKatla(x);
    printf("main icinde: %d\n", x);  // 10 — Orijinal değer değişmedi
    return 0;
}
```

4.4 Call by Value — Bellek Modeli (RAM Görünümü)

Aşağıdaki şema, `ikiyeKatla(x)` çağrısı sırasında bellekteki durumu göstermektedir:

```
RAM Bellek Durumu (Çağrı Anında):
┌──────────── main() ────────────┐   ┌──────────── ikiyeKatla() ──────────┐
│ Adres: 0x1000                  │   │ Adres: 0x2000                      │
│                                │   │                                    │
│  ┌──────┐                      │   │  ┌──────────┐                     │
│  │ x = 10 │ ──(Kopyalama)──────┼──→│  │sayi = 10 │ → sayi = 20         │
│  └──────┘                      │   │  └──────────┘                     │
│                                │   │                                    │
│ x değişmedi. Hâlâ 10.          │   │ Fonksiyon bitince SİLİNİR.         │
└────────────────────────────────┘   └────────────────────────────────────┘
```

Gözlem: `sayi` değişkeni, `x`'in RAM'deki farklı bir adrese yazılmış kopyasıdır. Fonksiyon sonlandığında bu kopya bellekten kaldırılır; `x` ise olduğu yerde aynen kalır.

⚠️ **Sonraki Hafta (Bellek Yönetimi):** Orijinal veriyi fonksiyon içinden değiştirmek gerektiğinde **Pointer** (işaretçi) kullanılır.

4.5 Call by Value — İzleme Tablosu

`ikiyeKatla(x)` çağrısının adım adım izlenmesi:

Adım	Konum	İşlem	`x` (main)	`sayi` (fonksiyon)
1	`main`	`x = 10` tanımlandı	10	—
2	`main → ikiyeKatla`	`x`'in kopyası `sayi`'ye yazıldı	10	10
3	`ikiyeKatla`	`sayi = sayi * 2` çalıştırıldı	10	20
4	`ikiyeKatla → main`	Fonksiyon sonlandı, `sayi` silindi	10	*(silindi)*

Sonuç: `x`'in değeri, fonksiyon çağrısı boyunca **hiç değişmemiştir.**

4.6 `return` Deyiminin Semantiği

`return` deyimi iki temel işlev üstlenir:

1. Hesaplanan değeri **çağıran bağlama** (çoğunlukla `main`) iletir.
2. Fonksiyonun yürütülmesini **anında** sonlandırır. `return` satırından sonraki kodlar **asla çalıştırılmaz** (dead code).

```c
float voltajDegeri(int adc_ham) {
    float sonuc = (adc_ham * 5.0) / 1023.0;   // ADC → Volt dönüşümü

    return sonuc;   // Değeri çağırana ilet ve fonksiyondan çık

    printf("Sensor okundu.");  // ❌ Ölü kod — asla çalıştırılmaz
}

int main() {
    float batarya = voltajDegeri(512);  // ≈ 2.503 V
    printf("Batarya: %.3f V\n", batarya);
    return 0;
}
```

4.7 İzleme Tablosu — `voltajDegeri(512)` Çağrısı

`voltajDegeri(512)` çağrısı sırasında bellekteki değişimlerin izleme tablosu:

Adım	Konum	İşlem	`adc_ham`	`sonuc`	`batarya`
1	`main`	`batarya` henüz tanımsız	—	—	?
2	`main → voltajDegeri`	Argüman 512 kopyalandı	512	—	?
3	`voltajDegeri`	Formül hesaplandı	512	≈2.503	?
4	`voltajDegeri → main`	`return sonuc` çalıştı	*(silindi)*	*(silindi)*	≈2.503

Gözlem: Fonksiyon sonlandığında `adc_ham` ve `sonuc` değişkenleri bellekten tamamen kaldırılır (yerel kapsam kuralı). Yalnızca `return` ile iletilen değer, çağıran taraftaki `batarya` değişkenine atanır.

4.8 `main()` Fonksiyonunun Dönüş Değeri

`main()` da bir fonksiyondur ve `int` tipinde değer döndürür. Bu değer **işletim sistemine** (çekirdek) veya gömülü sistemlerdeki **başlatıcı yazılıma** (bootloader/script) iletilir:

Dönüş Değeri	Anlamı	Sabit Karşılığı (`stdlib.h`)
`return 0;`	Program **başarıyla** tamamlandı	`EXIT_SUCCESS`
`return 1;` (veya ≠0)	Program **hatayla** sonlandı	`EXIT_FAILURE`

4.8.1 Uygulama: Hata Kodlarıyla Program Sonlandırma

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int mesafe;
    printf("Lutfen guvenli mesafe girin (cm): ");
    scanf("%d", &mesafe);

    if (mesafe <= 0) {
        printf("HATA: Gecersiz mesafe girdisi!\n");
        return EXIT_FAILURE;   // İşletim sistemine hata bildirimi (1)
    }

    printf("Sistem calisiyor. Mesafe: %d cm\n", mesafe);
    return EXIT_SUCCESS;       // Başarı bildirimi (0)
}
```

⚠️ **Mekatronik Bağlamı:** Otomasyon hattında çalışan bir kontrol programı `EXIT_FAILURE` döndürürse, sistemi denetleyen üst yazılım bu hatayı algılayarak hattı durdurabilir.

4.9 `void` Fonksiyonlarda Erken Çıkış (Guard Clause Deseni)

`void` fonksiyonlarda `return;` (değersiz) deyimi, fonksiyonun yürütülmesini anında sonlandırmak için kullanılır. Bu yaklaşım, iç içe `if-else` yapılarını düzleştirerek kodun okunabilirliğini artırır.

```
1   void motorAciAyarla(int aci) {
2       // Güvenlik kontrolü (Guard clause)
3       if (aci < 0 || aci > 180) {
4           printf("HATA: Servo motor 0-180 derece araliginda calisir!\n");
5           return;   // Geçersiz girdi — fonksiyondan derhal çık
6       }
7
8       // Bu noktaya yalnızca geçerli açı değerleriyle ulaşılır
9       printf("Servo motor %d dereceye konumlandiriliyor...\n", aci);
10  }
11
12  int main() {
13      motorAciAyarla(90);    // Geçerli — çalışır
14      motorAciAyarla(200);   // Geçersiz — uyarı verir ve çıkar
15      motorAciAyarla(-10);   // Geçersiz — uyarı verir ve çıkar
16      return 0;
17  }
```

4.10 İç İçe Fonksiyon Çağrıları (Nested Calls)

Bir fonksiyon başka bir fonksiyonu, o da üçüncü bir fonksiyonu çağırabilir. Bu mekanizma, karmaşık görevlerin katmanlı biçimde çözümlenmesini sağlar.

```c
float celsiusOku(void) {
    return 36.5;   // Sensör simülasyonu
}

float fahrenheitCevir(float c) {
    return (c * 9.0 / 5.0) + 32.0;
}

void sicaklikRaporla(void) {
    // Çağrı zinciri: sicaklikRaporla → fahrenheitCevir → celsiusOku
    float derece = fahrenheitCevir(celsiusOku());
    printf("Sicaklik: %.1f F\n", derece);
}

int main() {
    sicaklikRaporla();   // Tek çağrı ile 3 fonksiyon tetiklenir
    return 0;
}
```

4.11 Çağrı Yığını (Call Stack) Görünümü

İşlemci, iç içe çağrıları **Çağrı Yığını (Call Stack)** üzerinde **LIFO** (Son Giren İlk Çıkar) düzeninde izler:

```
╔═══════════════════════╗
║═══════════════════════╕
  3. celsiusOku()       ← En üstte (ilk biter)
╒═══════════════════════╕
  2. fahrenheitCevir()  ← Bekliyor
╒═══════════════════════╕
  1. sicaklikRaporla()  ← Bekliyor
╒═══════════════════════╕
  0. main()             ← En altta (son biter)
╚═══════════════════════╝
```

Çözülme sırası:

1. `celsiusOku()` → 36.5 döndürür → Yığından çıkar
2. `fahrenheitCevir(36.5)` → 97.7 döndürür → Yığından çıkar
3. `sicaklikRaporla()` → Ekrana yazar → Yığından çıkar
4. `main()` → Program sonlanır

BÖLÜM 5

Fonksiyon Bildirimi (Prototip) ve Kapsam Kuralları

5.1 Derleyicinin Yukarıdan Aşağı Okuma Kuralı

C derleyicisi kaynak kodu **yukarıdan aşağıya** doğru tek geçişte (single-pass) işler. Bir fonksiyon, tanımlanmadan önce çağrılırsa derleyici bu tanımlayıcıyı tanımaz ve hata üretir.

× Hatalı Sıralama:

```
int main() {
    float sonuc = topla(5, 10);   // Hata: 'topla' tanınmıyor
    return 0;
}

float topla(float a, float b) {
    return a + b;
}
```

5.2 Fonksiyon Prototipleri (Forward Declaration)

Çözüm, fonksiyonun **prototipini** (imza bildirimini) `main()` öncesine yerleştirmektir. Prototip, derleyiciye fonksiyonun varlığını, dönüş tipini ve parametre listesini önceden bildirir.

✅ Doğru Yaklaşım — Prototip Kullanımı:

```c
#include <stdio.h>

// Prototip — fonksiyonun imzası (noktalı virgül ile sonlanır)
// Not: #include <stdio.h> gibi satirlar aslinda kütüphane fonksiyonlarinin
// hazir prototiplerini dosyamiza dahil eder.
float topla(float a, float b);

int main() {
    float sonuc = topla(5, 10);   // Derleyici artık 'topla'yı tanır
    printf("Sonuc: %.1f\n", sonuc);
    return 0;
}

// Tanımlama — fonksiyonun gövdesi (noktalı virgül YOKTUR)
float topla(float a, float b) {
    return a + b;
}
```

5.3 Prototip ve Tanımlama — Karşılaştırma

Özellik	Prototip (Declaration)	Tanımlama (Definition)
Konum	`main()` öncesinde (üst kısımda)	`main()` sonrasında (alt kısımda)
Sonlandırma	Noktalı virgül (`;`) ile biter	Süslü parantez (`{ }`) ile biter
Gövde	Gövde yoktur	Gövde vardır
Amacı	Derleyiciye "bu fonksiyon var" der	Fonksiyonun gerçek iş mantığını içerir
Zorunluluk	Fonksiyon `main` sonrasında tanımlandıysa zorunlu	Her zaman gereklidir

```
// Prototip:  float topla(float a, float b);     ← noktalı virgül, gövde YOK
// Tanımlama: float topla(float a, float b) { return a + b; }  ← gövde VAR
```

5.4 Kapsam (Scope) — Değişkenlerin Yaşam Süresi

Bir değişken, tanımlandığı süslü parantez bloğu (`{ }`) içinde **doğar**, **yaşar** ve blok sonlandığında bellekten **kaldırılır**. Bu kurala **yerel kapsam (local scope)** denir.

```
1   void sensorOku() {
2       int veriler = 1500;   // Yalnızca bu blokta geçerlidir
3   }
4
5   int main() {
6       sensorOku();
7
8       // ❌ Derleme hatası: 'veriler' bu kapsamda tanımlı değil
9       // printf("%d", veriler);
10      return 0;
11  }
```

5.5 Stack (Yığın) Bellek Bölgesi — Tanım

Bir C programı çalıştırıldığında RAM belleği birden fazla bölgeye ayrılır. Fonksiyonların yerel değişkenleri ve çağrı bilgileri, **Stack (Yığın)** adı verilen özel bir bellek bölgesinde tutulur.

Bellek Bölgesi	İçeriği	Yaşam Süresi
Code (Text)	Derlenmiş makine kodları	Program boyunca sabit
Global / Static	Global değişkenler, `static` değişkenler	Program boyunca sabit
Stack (Yığın)	Yerel değişkenler, fonksiyon parametreleri, dönüş adresleri	Fonksiyon çağrısı süresince
Diğer Alanlar	Dinamik bellek vb. (Gelecek konu)	Değişken

Yüksek Adresler (0xFFFFFFFF)

Stack (Yığın) ⬇️

Fonksiyon parametreleri, yerel değişkenler, dönüş adresleri.

Aşağı doğru büyür.

⬍ Boş Bellek Alanı ⬍

Diğer bellek bölgeleriyle olan tampon bölge.

Global / Static Veriler

Program boyunca yaşayan değişkenler.

Code (Text) Segmenti

Makine kodları. Salt okunurdur.

Düşük Adresler (0x00000000)

Detaylı Analiz

- Stack Yönetimi: Otomatik yönetilir. Değişkenler oluşturulur ve silinir.
- LIFO: *"Son Giren İlk Çıkar"* prensibi.
- Gezinme: Fonksiyon bittiğinde bellek anında iade edilir.

5.6 Kapsam — Bellek Yaşam Döngüsü Gösterimi

Fonksiyon çağrıldığında yerel değişkenler **Stack'te** oluşturulur; fonksiyon sonlandığında **otomatik olarak** kaldırılır:

```
Zaman ──────────────────────────────────────────────────────►

          sensorOku()                    sensorOku()
          çağrıldı                       sonlandı
             │                              │
             ▼                              ▼
Stack:  ┌──────────┐                   ┌──────────┐
        │veriler=  │                   │          │
        │  1500    │  ← var            │ (boş)    │  ← silindi (pop)
        └──────────┘                   └──────────┘
```

Kural: Bir fonksiyonun yerel değişkenlerine, fonksiyon dışından **asla erişilemez.** Stack'teki alan fonksiyon bittiğinde geri alınır; bu mekanizma fonksiyonlar arası veri güvenliğini garanti eder.

5.7 İsim Gölgeleme (Shadowing) ve Kapsam Yalıtımı

Yerel kapsam kuralı sayesinde farklı fonksiyonlarda **aynı ada sahip** değişkenler tanımlanabilir. Her fonksiyon yalnızca kendi kapsamındaki değişkeni görür.

```c
void motorSur() {
    float hiz = 10.5;    // motorSur kapsamında
    printf("Motor Hizi: %.1f\n", hiz);
}

void fanKontrol() {
    float hiz = 80.0;    // fanKontrol kapsamında — farklı bir değişken
    printf("Fan Hizi: %.1f\n", hiz);
}

int main() {
    motorSur();      // 10.5 yazdırır
    fanKontrol();    // 80.0 yazdırır
    return 0;
}
// İsim çakışması oluşmaz; her fonksiyon yalnızca kendi yerel değişkenini kullanır.
```

5.8 Blok İçi Kapsam (Döngü ve Koşul Yapılarında)

Kapsam kuralları fonksiyonlarla sınırlı değildir. `for`, `while`, `if` gibi kontrol yapılarının süslü parantezleri de kendi alt kapsamlarını oluşturur.

```c
int main() {
    // 'i' yalnızca for döngüsü bloğunda geçerlidir
    for (int i = 0; i < 5; i++) {
        int temp = i * 10;
        printf("i=%d, temp=%d\n", i, temp);
    }

    // ❌ 'i' ve 'temp' bu noktada tanımsızdır — blok sonlandığında silindiler
    // printf("%d %d", i, temp);

    // ✅ Döngü sonrasında da 'j'ye erişim gerekiyorsa, dış kapsamda tanımlayın:
    int j;
    for (j = 0; j < 5; j++) { /* ... */ }
    printf("Son j: %d\n", j);   // Geçerli — j, main bloğunda tanımlı

    return 0;
}
```

⚠️ **İyi Pratik:** Değişkenleri olabildiğince dar kapsamda tanımlayın. Gereksiz geniş kapsam, isim çakışması ve

5.9 Global Değişkenler ve Riskleri

Herhangi bir bloğun (`{ }`) dışında, dosya düzeyinde tanımlanan değişkenler **global kapsama** sahiptir. Programdaki tüm fonksiyonlar bu değişkene erişebilir ve değerini değiştirebilir.

✅ **Avantajı:** Veriye her yerden kolayca erişilebilir. × **Dezavantajı:** Hangi fonksiyonun ne zaman değeri değiştirdiğini takip etmek zorlaşır (bakım güçlüğü, **verilerin kontrolsüz ezilmesi riski**).

```
int sistemHataKodu = 0;   // GLOBAL — tüm fonksiyonlar erişebilir

void sensor1Oku() {
    sistemHataKodu = 5;    // Değeri ezdi
}
void sensor2Oku() {
    sistemHataKodu = 10;   // Tekrar ezdi — önceki değer kayboldu
}

int main() {
    sensor1Oku();
    sensor2Oku();
    // sistemHataKodu'nun son değeri tahmine dayalı hale gelir.
    return 0;
}
```

5.10 Yerel ve Global Değişken — Özet Karşılaştırma

Özellik	Yerel (Local)	Global
Tanım Yeri	Fonksiyon/blok içinde `{ }`	Tüm fonksiyonların dışında
Yaşam Süresi	Blok başlangıcından sonuna kadar	Programın başından sonuna kadar
Görünürlük	Yalnızca tanımlandığı blok	Tüm fonksiyonlar
Başlangıç değeri	Çöp değer (garbage) — atanmalı	Otomatik olarak 0
Güvenlik	✅ Yüksek — izole	× Düşük — herkes erişir
Tavsiye	✅ Tercih edilir	⚠️ Donanımsal zorunluluklar dışında kaçınılmalı

Tasarım İlkesi: Çok kritik donanımsal senaryolar haricinde, gömülü yazılımda **global değişken kullanımından kaçınılmalıdır.** Veriler, fonksiyonlara parametre olarak aktarılmalıdır.

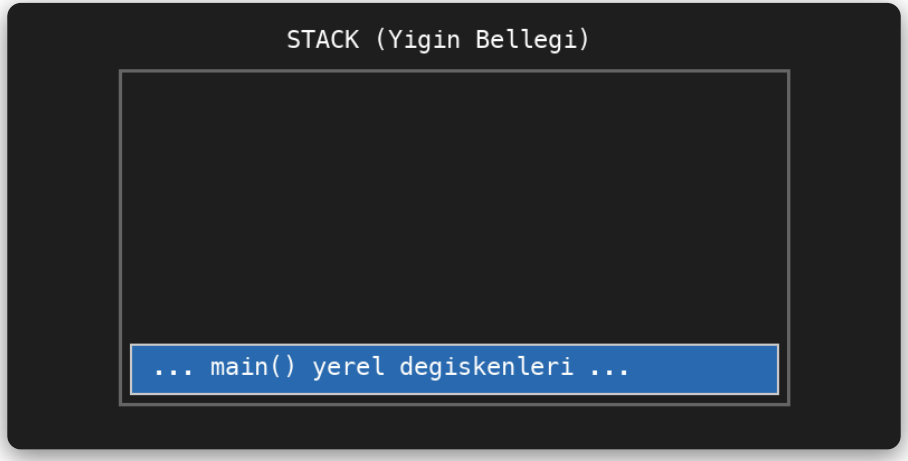
5.11 Fonksiyon Çağrısının Maliyeti (Overhead) — Ön Bilgi

Her fonksiyon çağrısında işlemci aşağıdaki adımları gerçekleştirir:

1. Parametreleri **Stack'e (Yığın Belleğine) yaz** (push)
2. **Dönüş adresini kaydet** (fonksiyon bittiğinde hangi satıra dönüleceğini sakla)
3. Program sayacını fonksiyonun gövdesine **yönlendir** (jump)
4. İş tamamlanınca **Stack'den geri yükle** (pop) ve çağıran noktaya dön

5.12 Stack (Yığın) Belleği — Detaylı Gösterim

Fonksiyon çağrıldığında Stack'te neler olur:



Bu maliyet çoğu durumda **ihmal edilebilir** düzeydedir. Ancak bir sensörün saniyede milyonlarca kez okunduğu gerçek zamanlı sistemlerde bu gecikme kritik hale gelebilir.

👉 **Hafta 9'da ele alınacak:** `inline` fonksiyonlar ve `#define` makrolarla bu maliyetin ortadan kaldırılması.

BÖLÜM 6

Sık Karşılaşılan Hatalar ve Çözümleri

6.1 Derleme Hatası Kataloğu (Compilation Errors)

Laboratuvar ortamında en sık karşılaşılan fonksiyon kaynaklı derleme hataları:

#	Hata Mesajı (GCC)	Sebep	Çözüm
1	`implicit declaration of function`	Fonksiyon prototipi yazılmamış	Dosyanın başına prototip ekleyin
2	`too few arguments to function`	Beklenen parametre sayısından az argüman	Çağrıdaki argüman sayısını kontrol edin
3	`too many arguments to function`	Gereğinden fazla argüman gönderilmiş	Fonksiyon tanımıyla eşleştirin
4	`void value not ignored`	`void` fonksiyon sonucu atanmaya çalışılmış	`void` fonksiyondan `=` ile değer almayın
5	`control reaches end of non-void`	Değer döndürmesi gereken fonksiyonda `return` eksik	Her yola `return` ekleyin
6	`conflicting types`	Prototip ile tanımlama uyumsuz	İkisini birebir aynı yapın

6.2 Derleme Hataları — Kod Örnekleri

```c
// HATA 1: Prototip unutulmuş
int main() {
    int s = topla(3, 5);   // ❌ 'topla' tanınmıyor
    return 0;
}
int topla(int a, int b) { return a + b; }
// ✅ Çözüm: "int topla(int a, int b);" satırını main'den önce ekle

// HATA 4: void fonksiyon sonucu atanmış
void selamla() { printf("Merhaba!\n"); }
int main() {
    int x = selamla();   // ❌ void fonksiyondan değer alınamaz
    return 0;
}

// HATA 5: return eksik
int kare(int n) {
    int sonuc = n * n;
    // ❌ return yazılmamış — derleyici uyarısı
}
```

6.3 Mantık Hatası Kataloğu (Logic Errors)

Derleyici hata vermez ancak program beklendiği gibi çalışmaz:

#	Hata	Belirtisi	Çözüm
1	`return` sonrası kod yazmak	İlgili satırlar asla çalışmaz	`return` deyimini mantıksal sona koyun
2	Call by Value'yu göz ardı etmek	"Değer neden değişmedi?"	Pointer kullanımı (sonraki hafta)
3	Global değişkeni farklı fonksiyonlarda ezmek	Rastgele değer karmaşası	Parametre ile veri aktarın
4	Prototip — tanım uyumsuzluğu	Beklenmeyen dönüş değerleri	İmzaları birebir eşleyin

```
// ❌ Yaygın mantık hatası: return sonrası ölü kod
int hesapla(int x) {
    return x * 2;
    x = x + 10;   // Bu satır ASLA çalıştırılmaz (Dead Code)
}
```

BÖLÜM 7

Uygulamalı Laboratuvar Çalışmaları

7.1 Uygulama 1: Robot Yön Kontrolü

Derste öğrenilen `void` fonksiyon yapısını pekiştirmek amacıyla basit bir robot hareket senaryosu:

```c
#include <stdio.h>

void ileri(void)  { printf(">> Robot ILERI gidiyor...\n"); }
void geri(void)   { printf("<< Robot GERI gidiyor...\n"); }
void dur(void)    { printf("|| Robot DURDU.\n"); }

int main() {
    ileri();
    ileri();
    geri();
    dur();
    return 0;
```

Beklenen Çıktı:

```
>> Robot ILERI gidiyor...
>> Robot ILERI gidiyor...
<< Robot GERI gidiyor...
|| Robot DURDU.
```

7.2 Uygulama 2: Sıcaklık Dönüştürücü (Celsius → Fahrenheit)

Parametre alan ve değer döndüren fonksiyon ile prototip kullanımını pekiştirme:

```c
#include <stdio.h>

float celsiusToFahrenheit(float c);   // Prototip

int main() {
    float derece;
    printf("Sicaklik (C): ");
    scanf("%f", &derece);

    float sonuc = celsiusToFahrenheit(derece);
    printf("%.1f C = %.1f F\n", derece, sonuc);

    if (sonuc > 100.0) {
        printf("UYARI: Motor sogutma sistemi aktif edilmeli!\n");
    }
    return 0;
}

float celsiusToFahrenheit(float c) {
```

7.3 Uygulama 3: Tekerlek Geometrisi Hesabı

Birden fazla fonksiyonun koordineli çalışması ve `#define` ile sabit tanımlamayı pekiştirme:

```c
#include <stdio.h>
#define PI 3.14159265

float alanHesapla(float r);      // Prototip
float cevreHesapla(float r);     // Prototip

int main() {
    float yaricap;
    printf("Tekerlek yaricapi (cm): ");
    scanf("%f", &yaricap);

    printf("Alan  : %.2f cm2\n", alanHesapla(yaricap));
    printf("Cevre : %.2f cm\n", cevreHesapla(yaricap));
    printf("1 tur : %.2f cm yol alir.\n", cevreHesapla(yaricap));
    return 0;
}

float alanHesapla(float r) {
    return PI * r * r;
```

7.4 Uygulama 4: Sensör Veri Sınıflandırıcı

Kullanıcıdan okunan ADC değerine göre voltaj hesabı yapan ve durumu sınıflandıran çok fonksiyonlu program:

```c
#include <stdio.h>

float adcToVoltaj(int adc);             // ADC → Volt
void durumRaporla(float voltaj);        // Voltaja göre rapor

int main() {
    int adc_degeri;
    printf("ADC degeri (0-1023): ");
    scanf("%d", &adc_degeri);

    if (adc_degeri < 0 || adc_degeri > 1023) {
        printf("HATA: Gecersiz ADC araligi!\n");
        return 1;   // EXIT_FAILURE
    }

    float volt = adcToVoltaj(adc_degeri);
    printf("Voltaj: %.3f V\n", volt);
```

Pekiştirilen Kavramlar: ✅ Girdi doğrulama, `return 1` ile hata çıkışı, fonksiyon zinciri, Guard Clause.

Özet ve Sonraki Ders

Bu Hafta İşlenen Konular:

- **Modüler Tasarım:** Fonksiyonlar, kodun yönetilebilir alt birimlere ayrılmasını sağlar.
- **Anatomi:** Geri dönüş tipi, parametre listesi, gövde ve `return` deyimi. Dört fonksiyon çeşidi.
- **Call by Value:** Fonksiyona argüman olarak gönderilen değişkenin kopyası oluşturulur; orijinal etkilenmez.
- `void` **ve Erken Çıkış:** `void` fonksiyonlar değer döndürmez; `return;` ile Guard Clause uygulanabilir.
- `main()` **Dönüş Değeri:** `return 0` başarı, `return 1` hata bildirir.
- **Call Stack:** İç içe çağrılarda LIFO düzeninde yığın yönetimi.
- **Prototip:** Derleyiciye fonksiyon imzasının önceden bildirilmesi.
- **Kapsam (Scope):** Yerel, blok-içi ve global kapsam kuralları.
- **Overhead:** Her fonksiyon çağrısı Stack'te yer kaplar (Hafta 9: `inline` çözümü).

Sonraki Ders — Ön Bakış

Açık Kalan Soru:

> Bir fonksiyondan `return` ile yalnızca **tek bir değer** döndürülebilir. Peki bir motor kontrol fonksiyonunun aynı anda konum, hız ve sıcaklık gibi **birden fazla değeri** güncellemesi gerekirse ne yapılacaktır?

👉 Çözüm Gelecek Hafta: C Dilinin en güçlü mekanizmalarından biri olan İşaretçiler (Pointers) ve Bellek Yönetimi.

Teşekkürler...

Dr. Öğr. Üyesi Hüseyin YÜCE

Marmara Üniversitesi

Mekatronik Mühendisliği Bölümü